

Group Project Proposal

Improving Repository Navigation and Bug Localization in Minimal Software Engineering Agents

Group ID: 25

DSA4213: Natural Language Processing for Data Science
Semester 1, AY2026/2027

Administrative Information

Group members	ZHANG YUHAN(A0351628B) WANG CHI(A0357310M) CHEN WENWEN(A0351705J) JIANG ZHANGZHANG(A0355183B) CHEN XINGYU(A0355370E)
Project direction	Software Engineering Agents – Repository Navigation and Bug Localization.
Repository	https://github.com/235559649/DSA4213_group_work

Project Summary

Software engineering agents such as mini-SWE-agent can autonomously explore repositories, modify code, and execute tests, but repository exploration is largely driven by free-form shell interaction. This project studies whether structured repository navigation and failure-aware bug localization can improve the efficiency and reliability of software repair agents. We will extend mini-SWE-agent with a Python-oriented repository indexer that ranks relevant files and functions based on issue descriptions, code structure, and runtime failure evidence. We will compare the proposed agent with the original mini-SWE-agent on real bug-fixing tasks, evaluating repair success rate, localization accuracy, number of explored files, shell commands, token usage, runtime, and failure patterns.

1. Problem, Research Question, and Motivation

TODO: CHEN WENWEN

Software engineering agents must often explore a large repository before they can determine which files or functions are responsible for a reported bug. A minimal agent such as mini-SWE-agent can perform this exploration using shell commands and language-model reasoning, but the navigation process is not explicitly optimized for locating relevant code.

Our project focuses on the repository exploration and bug localization stage of the software repair process. Rather than replacing the existing repair capability of mini-SWE-agent, we investigate whether additional repository structure and runtime failure evidence can help the agent reach relevant code more efficiently.

TODO: Add a short motivating example showing an issue where the agent must search through several files before reaching the correct function.

1.1 Primary Research Question

RQ1. Can structured repository navigation and runtime failure evidence help mini-SWE-agent localize relevant code more efficiently without reducing bug-fixing success?

1.2 Secondary Research Questions

RQ2. Does improved localization reduce repository exploration cost, measured by the number of files inspected, shell commands executed, token usage, and runtime?

RQ3. Which information sources contribute most to successful localization: textual relevance to the issue, static repository structure, or runtime failure evidence?

1.3 Definition of Success

The project will be considered successful if the proposed system can either improve bug localization and repair performance, or achieve comparable repair performance while reducing unnecessary repository exploration.

TODO: Define quantitative success criteria after pilot experiments, for example an improvement in Top- k localization accuracy or a reduction in exploration steps.

2. Related Work and Proposed Contribution

TODO: WNAG CHI

2.1 mini-SWE-agent

mini-SWE-agent will serve as the primary baseline and software repair agent in this project.

TODO: Describe the mini-SWE-agent architecture, its minimal agent loop, available tools, and how repository exploration is performed. Add the appropriate citation.

TODO: Clarify exactly which parts of mini-SWE-agent are reused without modification and which integration points will be changed by our system.

2.2 SWE-bench

SWE-bench provides real-world software engineering tasks based on repository issues and corresponding code changes.

TODO: Describe the chosen SWE-bench version/subset and justify why it is appropriate for evaluating our project. Add citation.

TODO: Specify whether the project will use SWE-bench Verified or another subset, and how many tasks will be used for development and evaluation.

2.3 Repository Localization Methods

Previous software engineering agents and code-retrieval systems have used techniques such as lexical search, semantic retrieval, repository structure, dependency information, and execution feedback to identify code relevant to a software issue.

TODO: Review and cite the most relevant repository navigation / bug localization systems. Include systems that are particularly close to our proposed approach so that our novelty is clearly stated.

TODO: Explain how our approach differs from prior work rather than only listing related systems.

2.4 Our Contribution

- C1.** A Python-oriented repository indexer that extracts structural information at the file, class, and function levels and supports task-aware localization of relevant code.
- C2.** A failure-aware localization mechanism that uses runtime evidence, including tracebacks and failing tests, to refine previously ranked suspicious code locations.
- C3.** An empirical study comparing repair performance and repository exploration efficiency against the original mini-SWE-agent baseline.

TODO: Finalize the exact ranking algorithm before submission. For example, decide whether C1 combines keyword similarity, semantic similarity, repository structure, or another scoring method.

3. Agent and Task Setup

TODO: ZHANG YUHAN

3.1 Task

The agent receives a software issue together with a Python repository. Its goal is to identify the code relevant to the issue, produce a patch, and validate the patch through execution or tests.

Input: a natural-language issue description and the corresponding repository.

Output: a proposed source-code patch and an execution trajectory containing repository exploration, edits, and test results.

3.2 Operating Environment

The repair agent will operate inside an isolated execution environment with access to the repository, shell commands, Python execution, Git, and the repository’s available test framework.

TODO: Specify the exact environment, e.g., Docker / SWE-bench containers, Python versions, and available system tools.

3.3 Agent and Model

The original mini-SWE-agent will be used as the software repair baseline. Our system will augment its repository exploration process with structured localization information.

TODO: Specify the LLM(s) used in experiments.

TODO: Specify whether all compared systems will use the same model, temperature, token budget, and prompts to ensure a controlled comparison.

3.4 Agent Tools

The proposed system may provide the agent with the following sources of information:

- shell-based repository exploration;
- file, class, and function indexing;
- Python AST and import information;
- issue-to-code relevance scores;
- test execution results; and
- traceback and runtime failure evidence.

TODO: Remove any tool that is not implemented in the final system and describe the exact interface exposed to the agent.

3.5 Benchmark and Task Selection

The initial study will focus on Python repositories so that Python ASTs, tracebacks, and testing tools can be exploited consistently.

TODO: Define the exact benchmark subset and task selection criteria.

TODO: Specify the number of tasks used for pilot development and final evaluation.

3.6 Stopping Conditions and Failure Recovery

An agent run may terminate when the target tests pass, when the agent explicitly submits a final patch, or when a predefined resource budget is exhausted.

When execution fails, the system can extract information from the failure and use it to update the localization ranking before the next repair attempt.

TODO: Define maximum agent steps, token budget, runtime limit, and the exact condition used to determine successful repair.

4. Proposed System or Study Design

TODO: CHEN XINGYU

The proposed workflow consists of an initial repository analysis stage, a localization stage, the existing mini-SWE-agent repair process, and an optional refinement stage based on runtime failures.

TODO: Insert system architecture diagram: Issue → Repository Index → Initial Localization → mini-SWE-agent → Test Execution → Failure-Aware Refinement → Repair.

Figure 1: Proposed system architecture.

4.1 Baseline

The baseline is the original mini-SWE-agent operating with its standard repository exploration and repair loop.

TODO: Describe the exact baseline configuration and ensure it uses the same LLM and execution budget as the proposed system.

4.2 Repository Indexing

Before repair begins, the repository indexer analyzes the Python repository and records structural information such as:

- file paths;
- classes;
- functions and methods;
- import relationships; and
- selected code metadata relevant to localization.

TODO: Define the actual internal representation. Decide whether this will be a table, graph, JSON index, or another structure.

TODO: Decide whether call relationships or inheritance information are included. Avoid adding them unless they are actually used.

4.3 Initial Localization

Given an issue description, the localization module ranks repository files and functions according to their estimated relevance to the issue.

The result will be a ranked list of suspicious locations that can be provided to the repair agent as additional context.

TODO: Define the localization scoring function.

TODO: Decide whether the initial implementation includes keyword search, semantic embeddings, structural signals, or a hybrid method.

4.4 Failure-Aware Refinement

If a reproduction attempt or test execution fails, the system extracts runtime evidence such as:

- exception type;
- traceback frames;
- files and functions appearing in the traceback;
- failing test names; and
- relevant error messages.

This information is then used to update the ranking of suspicious code locations.

TODO: Define how runtime evidence modifies the original ranking.

TODO: Provide one concrete example trajectory in the final proposal if space permits.

4.5 Repair Loop

The localized context is passed to mini-SWE-agent, which investigates the repository, modifies source code, and executes tests. If the patch fails, failure information may be returned to the localization component and a new repair attempt can be made.

TODO: Specify whether localization is performed only once or repeatedly during the trajectory. This decision is important for the final system design.

4.6 Optional Extension: Reviewer Agent

If time permits, an independent reviewer agent may inspect a proposed patch before final submission.

The reviewer would focus on correctness, whether the reported failure has actually been addressed, potential regressions, and relevant edge cases.

TODO: Keep this as an optional extension. Do not make it part of the main claim unless the core localization system is completed early.

4.7 Comparisons and Ablations

The planned controlled conditions are:

- **Baseline A:** original mini-SWE-agent;
- **Baseline B:** mini-SWE-agent with simple lexical / keyword localization;
- **Method C:** mini-SWE-agent with repository-structure-aware localization;
- **Method D:** mini-SWE-agent with repository-structure-aware localization and failure-aware refinement.

Possible ablations will remove individual information sources from the full system.

TODO: Finalize comparison conditions after implementing a pilot version. Do not promise more baselines than the group can realistically run.

TODO: Possible ablations: no structural information, no failure evidence, or file-level rather than function-level localization.

5. Preliminary Evaluation Plan

We will evaluate the original mini-SWE-agent (A), lexical localization (B), structure-aware localization (C), and structure plus failure-aware refinement (D) on the same held-out SWE-bench Verified tasks. The primary comparison is A versus D; B and C indicate which localization components contribute to any difference. We will assess repair outcomes, localization quality, and exploration cost separately so that a faster search is not mistaken for a successful repair.

5.1 Repair Performance

The primary outcome is *instance resolution rate*: the proportion of selected tasks marked **resolved** by the official SWE-bench evaluation harness [2, 3]. A run submits one final patch. The harness applies it in the task environment and checks that all **FAIL_TO_PASS** tests pass while the **PASS_TO_PASS** tests remain passing. A missing or unapplicable patch, or a run that exhausts its budget, counts as unresolved. We will also report patch-application rate and the fractions of the two test groups passed, to distinguish incomplete fixes from regressions. Tests run by the agent during repair provide feedback but do not replace the official final evaluation.

5.2 Localization Performance

For offline scoring, the files changed by the benchmark’s reference patch define the ground-truth locations. We will include changed Python source files and exclude test, documentation, and generated files. For each ranked list, *Hit@1* and *Hit@5* indicate whether at least one ground-truth file appears in the first one or five positions; mean reciprocal rank uses the rank of the first such file. We will also report file Recall@5 because some fixes require changes in multiple files. Tasks without a changed Python source file will be excluded from file-localization scoring, with the exclusion count reported.

Initial rankings will be scored before the agent executes any command. For D, we will additionally compare rankings before and after the first usable test failure, reporting how many tasks actually produced such evidence. As a secondary analysis, changed lines will be mapped to functions in the task’s original Python AST; function Hit@5 will be reported only where this mapping is unambiguous, together with its task coverage. Reference patches and benchmark test labels will never be shown to the agent or used by its ranking method.

5.3 Exploration Efficiency

From saved action and observation trajectories, we will count unique Python source files whose contents the agent inspects and the number of model steps before it first inspects a ground-truth file. Runs that never reach such a file will be reported separately rather than assigned an artificial step count. We will also measure total input and output tokens and wall-clock time per task, including indexing and ranking overhead. Shell-command count and estimated API cost will be retained as supplementary measures. All added localization messages count toward the method’s token usage and cost.

5.4 Agent-Trajectory Analysis

We will inspect representative successes and failures using the same trajectory logs. A preliminary failure taxonomy assigns the main reason for each unsuccessful run to: (i) no relevant file inspected, (ii) relevant code inspected but the cause misdiagnosed, (iii) a patch that is missing, invalid, or incomplete, (iv) a regression or inadequate validation, or (v) an environment or resource-limit failure. Pilot runs will be used to refine coding rules before the held-out analysis. Case studies will show the candidate rankings, files inspected, test output, final patch, and official test result, including cases where localization improves but repair still fails.

5.5 Variability and Experimental Protocol

We provisionally plan approximately 20 development tasks and up to 100 disjoint held-out evaluation tasks from SWE-bench Verified, spanning multiple repositories. A pilot will establish the feasible task count and per-run budget; the final task list and protocol will be frozen before inspecting held-out results. All conditions will use identical task instances, the same pinned mini-SWE-agent and benchmark versions, the same LLM version and decoding settings, the same base prompts and execution environment, and equal step, token/cost, and wall-time limits. Only the prescribed localization context will differ. The exact model, temperature, limits, and applicable random seeds will be recorded after the pilot rather than assumed in advance.

The main study will submit one independent patch per task and condition. To assess run-to-run variability, we plan three independent runs per condition on a 20-task evaluation subset chosen before results are examined. Repeated runs will be reported individually and as averages, not combined into a best-of-three score. We will report paired task-level differences in resolution and efficiency, with confidence intervals where the final sample size permits. Infrastructure failures will be logged separately, with any exclusions disclosed.

6. Risks, Limitations, and Responsible Practice

TODO: CHEN WENWEN

- **LLM nondeterminism.** Repeated runs may produce different trajectories and repair results. We will use controlled model settings and repeated evaluation where resources permit.
- **Benchmark contamination.** Public repositories or issue solutions may have appeared in model training data, so benchmark performance should not be interpreted purely as unseen-code generalization.
- **Execution safety.** Software engineering agents can execute shell commands. Experiments will therefore be conducted in isolated environments with restricted permissions.
- **API and compute cost.** Full agent trajectories can require substantial token usage and execution time. A fixed experimental budget will be used.
- **Environment failures.** Some benchmark repositories may fail because of dependency or setup problems unrelated to the agent itself. These cases will be logged separately where possible.

- **Python-specific scope.** The initial system focuses on Python repositories, so conclusions may not directly generalize to other programming languages.

6.1 Safeguards

Experiments will use isolated execution environments, restricted credentials and permissions, fixed resource budgets, and public benchmark repositories only. No private repositories, credentials, or confidential source code will be included[1].

TODO: Confirm the exact sandbox/container mechanism used by the final implementation.

References

- [1] John Doe. Sample article. *Journal of Examples*, 1:1–10, 2024.
- [2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024.
- [3] SWE-bench contributors. SWE-bench evaluation harness reference. Official documentation. Accessed 2026-09-25.

TODO: Create references.bib and add citations for mini-SWE-agent, SWE-bench, repository localization methods, and any additional systems used by the project.